

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет Кафедра «Вычислительная
техника»

ОТЧЕТ

по лабораторной работе №6
по курсу «Логика и основы алгоритмизации в инженерных задачах» на
тему «Определение характеристик графов»

Выполнили:
студенты группы 24ВВВЗ
Мамонтов Н.П.
Масюк А.Р.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., Деев М.В.

Пенза 2025

Цель работы – Освоение методов программной обработки унарных и бинарных операции над графами.

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M1, M2 смежности неориентированных помеченных графов G1, G2. Выведите сгенерированные матрицы на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

а) отождествления вершин

б) стягивания ребра

в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:

а) объединения $G = G1 \cup G2$

б) пересечения $G = G1 \cap G2$

в) кольцевой суммы $G = G1 \oplus G2$

Результат выполнения операции выведите на экран.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

#define MAX_VERTICES 10

// Генерация случайного графа
void generateRandomGraph(int matrix[MAX_VERTICES][MAX_VERTICES], int vertices)
{
    for (int i = 0; i < vertices; i++) {
        for (int j = i; j < vertices; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else {
                matrix[i][j] = matrix[j][i] = rand() % 2;
            }
        }
    }
}
```

```

    }
}

// Вывод матрицы смежности
void printMatrix(int matrix[MAX_VERTICES][MAX_VERTICES], int vertices) {
    printf("      ");
    for (int i = 0; i < vertices; i++) {
        printf("%2d ", i + 1);
    }
    printf("\n");
    for (int i = 0; i < vertices; i++) {
        printf("%2d: ", i + 1);
        for (int j = 0; j < vertices; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

// Отождествление вершин
void identifyVertices(int matrix[MAX_VERTICES][MAX_VERTICES], int* vertices, int
u, int v) {
    if (u < 0 || v < 0 || u >= *vertices || v >= *vertices || u == v) {
        printf("Ошибка: неверные номера вершин\n");
        return;
    }

    for (int i = 0; i < *vertices; i++) {
        matrix[u][i] = matrix[u][i] || matrix[v][i];
        matrix[i][u] = matrix[u][i];
    }

    for (int i = v; i < *vertices - 1; i++) {
        for (int j = 0; j < *vertices; j++) {
            matrix[i][j] = matrix[i + 1][j];
        }
    }
    for (int i = 0; i < *vertices - 1; i++) {
        for (int j = v; j < *vertices - 1; j++) {
            matrix[i][j] = matrix[i][j + 1];
        }
    }

    (*vertices)--;
    printf("Вершины %d и %d отождествлены в вершину %d\n", u + 1, v + 1, u + 1);
}

// Стягивание ребра
void contractEdge(int matrix[MAX_VERTICES][MAX_VERTICES], int* vertices, int u,
int v) {
    if (u < 0 || v < 0 || u >= *vertices || v >= *vertices || u == v) {
        printf("Ошибка: неверные номера вершин\n");
        return;
    }

    if (matrix[u][v] == 0) {

```

```

        printf("Ошибка: вершины %d и %d не соединены ребром\n", u + 1, v + 1);
        return;
    }

    identifyVertices(matrix, vertices, u, v);
    printf("Ребро (%d, %d) стянуто\n", u + 1, v + 1);
}

// Расщепление вершины
void splitVertex(int matrix[MAX_VERTICES][MAX_VERTICES], int* vertices, int v)
{
    if (v < 0 || v >= *vertices) {
        printf("Ошибка: неверный номер вершины\n");
        return;
    }

    if (*vertices >= MAX_VERTICES) {
        printf("Ошибка: достигнут максимальный размер графа\n");
        return;
    }

    int newVertex = *vertices;

    for (int i = 0; i <= newVertex; i++) {
        matrix[newVertex][i] = 0;
        matrix[i][newVertex] = 0;
    }

    for (int i = 0; i < *vertices; i++) {
        if (i != v && matrix[v][i] == 1) {
            if (rand() % 2 == 0) {
                matrix[newVertex][i] = matrix[i][newVertex] = 1;
                matrix[v][i] = matrix[i][v] = 0;
            }
        }
    }

    matrix[v][newVertex] = matrix[newVertex][v] = 1;

    (*vertices)++;
    printf("Вершина %d расщеплена на вершины %d и %d\n", v + 1, v + 1, newVertex
+ 1);
}

// Объединение графов
void unionGraphs(int G1[MAX_VERTICES][MAX_VERTICES], int
G2[MAX_VERTICES][MAX_VERTICES],
int result[MAX_VERTICES][MAX_VERTICES], int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            result[i][j] = G1[i][j] || G2[i][j];
        }
    }
}

// Пересечение графов
void intersectionGraphs(int G1[MAX_VERTICES][MAX_VERTICES], int
G2[MAX_VERTICES][MAX_VERTICES],

```

```

    int result[MAX_VERTICES][MAX_VERTICES], int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            result[i][j] = G1[i][j] && G2[i][j];
        }
    }
}

// Кольцевая сумма графов (XOR)
void ringSumGraphs(int G1[MAX_VERTICES][MAX_VERTICES], int
G2[MAX_VERTICES][MAX_VERTICES],
int result[MAX_VERTICES][MAX_VERTICES], int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            result[i][j] = G1[i][j] ^ G2[i][j];
        }
    }
}

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(NULL));
    int choice;
    int vertices;
    int M1[MAX_VERTICES][MAX_VERTICES], M2[MAX_VERTICES][MAX_VERTICES];
    int result[MAX_VERTICES][MAX_VERTICES];

    do {
        printf("\n=====\\n");
        printf("Лабораторная работа №6: Операции над графами\\n");
        printf("=====\\n");
        printf("1. Задание 1: Генерация двух графов\\n");
        printf("2. Задание 2.1: Унарные операции над графом\\n");
        printf("3. Задание 3.1: Бинарные операции над графами\\n");
        printf("0. Выход\\n");
        printf("Выберите действие: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                printf("\\n--- Задание 1: Генерация двух графов ---\\n");
                printf("Введите количество вершин в графах: ");
                scanf("%d", &vertices);

                if (vertices <= 0 || vertices > MAX_VERTICES) {
                    printf("Ошибка: количество вершин должно быть от 1 до %d\\n",
MAX_VERTICES);
                    break;
                }

                generateRandomGraph(M1, vertices);
                generateRandomGraph(M2, vertices);

                printf("\\nМатрица смежности графа G1:\\n");
                printMatrix(M1, vertices);

                printf("Матрица смежности графа G2:\\n");
                printMatrix(M2, vertices);
            }
        }
    } while (choice != 0);
}

```

```

        break;
    }

    case 2: {
        printf("\n--- Задание 2.1: Унарные операции ---\n");

        int G[MAX_VERTICES][MAX_VERTICES];
        int currentVertices;

        printf("Создать новый граф или использовать существующий G1?\n");
        printf("1. Создать новый случайный граф\n");
        printf("2. Использовать граф G1 из задания 1\n");
        int subChoice;
        scanf("%d", &subChoice);

        if (subChoice == 1) {
            printf("Введите количество вершин: ");
            scanf("%d", &currentVertices);
            if (currentVertices <= 0 || currentVertices > MAX_VERTICES) {
                printf("Ошибка: неверное количество вершин\n");
                break;
            }
            generateRandomGraph(G, currentVertices);
            printf("\nСгенерированный граф:\n");
            printMatrix(G, currentVertices);
        }
        else if (subChoice == 2) {
            if (vertices <= 0) {
                printf("Сначала выполните задание 1 для генерации графов\n");
                break;
            }
            currentVertices = vertices;
            for (int i = 0; i < vertices; i++) {
                for (int j = 0; j < vertices; j++) {
                    G[i][j] = M1[i][j];
                }
            }
            printf("\nИспользуется граф G1:\n");
            printMatrix(G, currentVertices);
        }
        else {
            printf("Неверный выбор\n");
            break;
        }

        printf("\nВыберите унарную операцию:\n");
        printf("1. Отождествление вершин\n");
        printf("2. Стыгивание ребра\n");
        printf("3. Расщепление вершины\n");
        scanf("%d", &subChoice);

        int u, v;
        switch (subChoice) {
            case 1:
                printf("Введите номера вершин для отождествления (например: 2 4): ");
                scanf("%d %d", &u, &v);

```

```

        identifyVertices(G, &currentVertices, u - 1, v - 1);
        printf("\nРезультат отождествления:\n");
        printMatrix(G, currentVertices);
        break;

    case 2:
        printf("Введите номера вершин ребра для стягивания (например: 1
3): ");

        scanf("%d %d", &u, &v);
        contractEdge(G, &currentVertices, u - 1, v - 1);
        printf("\nРезультат стягивания ребра:\n");
        printMatrix(G, currentVertices);
        break;

    case 3:
        printf("Введите номер вершины для расщепления: ");
        scanf("%d", &u);
        splitVertex(G, &currentVertices, u - 1);
        printf("\nРезультат расщепления вершины:\n");
        printMatrix(G, currentVertices);
        break;

    default:
        printf("Неверный выбор\n");
    }
    break;
}

case 3: {
    printf("\n--- Задание 3.1: Бинарные операции ---\n");

    if (vertices <= 0) {
        printf("Сначала выполните задание 1 для генерации графов\n");
        break;
    }

    printf("Используются графы G1 и G2 из задания 1\n");
    printf("G1:\n");
    printMatrix(M1, vertices);
    printf("G2:\n");
    printMatrix(M2, vertices);

    printf("\nВыберите бинарную операцию:\n");
    printf("1. Объединение  $G = G1 \cup G2$ \n");
    printf("2. Пересечение  $G = G1 \cap G2$ \n");
    printf("3. Кольцевая сумма  $G = G1 \oplus G2$ \n");
    int opChoice;
    scanf("%d", &opChoice);

    switch (opChoice) {
        case 1:
            unionGraphs(M1, M2, result, vertices);
            printf("\nРезультат объединения G1 и G2:\n");
            printMatrix(result, vertices);
            break;

        case 2:
            intersectionGraphs(M1, M2, result, vertices);

```

```

        printf("\nРезультат пересечения G1 и G2:\n");
        printMatrix(result, vertices);
        break;

    case 3:
        ringSumGraphs(M1, M2, result, vertices);
        printf("\nРезультат кольцевой суммы G1 и G2:\n");
        printMatrix(result, vertices);
        break;

    default:
        printf("Неверный выбор\n");
    }
    break;

}

case 0:
    printf("Завершение программы...\n");
    break;

default:
    printf("Неверный выбор. Попробуйте снова.\n");
}

} while (choice != 0);

return 0;
}

```

Работа программы

```

F:\2 курс\ЛОАВИЗ\labik6\Debug\labik6.exe

=====
Лабораторная работа №6: Операции над графами
=====
1. Задание 1: Генерация двух графов
2. Задание 2.1: Унарные операции над графом
3. Задание 3.1: Бинарные операции над графами
0. Выход
Выберите действие: _

```

Главное меню

Выберите действие: 1

--- Задание 1: Генерация двух графов ---

Введите количество вершин в графах: 10

Матрица смежности графа G1:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	0	0	0	0	1	0	0
2:	0	0	1	0	0	0	1	1	0	0
3:	1	1	0	1	1	0	0	0	0	0
4:	0	0	1	0	0	1	0	1	0	0
5:	0	0	1	0	0	1	1	1	1	1
6:	0	0	0	1	1	0	0	0	0	0
7:	0	1	0	0	1	0	0	1	1	1
8:	1	1	0	1	1	0	1	0	0	0
9:	0	0	0	0	1	0	1	0	0	1
10:	0	0	0	0	1	0	1	0	1	0

Матрица смежности графа G2:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	1	0	0	0	1	0	1
2:	0	0	0	1	1	1	1	0	1	0
3:	1	0	0	1	1	0	0	0	1	0
4:	1	1	1	0	0	0	1	1	0	1
5:	0	1	1	0	0	0	1	1	1	1
6:	0	1	0	0	0	0	1	1	0	0
7:	0	1	0	1	1	1	0	0	1	0
8:	1	0	0	1	1	1	0	0	1	0
9:	0	1	1	0	1	0	1	1	0	0
10:	1	0	0	1	1	0	0	0	0	0

Задание 1

```

Выберите действие: 2

--- Задание 2.1: Унарные операции ---
Создать новый граф или использовать существующий G1?
1. Создать новый случайный граф
2. Использовать граф G1 из задания 1
2

Используется граф G1:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  1  0  0  0  0  1  0  0
2:  0  0  1  0  0  0  1  1  0  0
3:  1  1  0  1  1  0  0  0  0  0
4:  0  0  1  0  0  1  0  1  0  0
5:  0  0  1  0  0  1  1  1  1  1
6:  0  0  0  1  1  0  0  0  0  0
7:  0  1  0  0  1  0  0  1  1  1
8:  1  1  0  1  1  0  1  0  0  0
9:  0  0  0  0  1  0  1  0  0  1
10: 0  0  0  0  1  0  1  0  1  0

Выберите унарную операцию:
1. Отождествление вершин
2. Стягивание ребра
3. Расщепление вершины
1
Введите номера вершин для отождествления (например: 2 4): 2 4
Вершины 2 и 4 отождествлены в вершину 2

Результат отождествления:
  1  2  3  4  5  6  7  8  9
1:  0  0  1  0  0  0  1  0  0
2:  0  0  1  0  1  1  1  0  0
3:  1  1  0  1  0  0  0  0  0
4:  0  0  1  0  1  1  1  1  1
5:  0  1  0  1  0  0  0  0  0
6:  0  1  0  1  0  0  1  1  1
7:  1  1  0  1  0  1  0  0  0
8:  0  0  0  1  0  1  0  0  1
9:  0  0  0  1  0  1  0  1  0

```

Задание 2.1

```

--- Задание 2.1: Унарные операции ---
Создать новый граф или использовать существующий G1?
1. Создать новый случайный граф
2. Использовать граф G1 из задания 1
2

Используется граф G1:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  1  0  0  0  0  1  0  0
2:  0  0  1  0  0  0  1  1  0  0
3:  1  1  0  1  1  0  0  0  0  0
4:  0  0  1  0  0  1  0  1  0  0
5:  0  0  1  0  0  1  1  1  1  1
6:  0  0  0  1  1  0  0  0  0  0
7:  0  1  0  0  1  0  0  1  1  1
8:  1  1  0  1  1  0  1  0  0  0
9:  0  0  0  0  1  0  1  0  0  1
10: 0  0  0  0  1  0  1  0  1  0

Выберите унарную операцию:
1. Отождествление вершин
2. Стягивание ребра
3. Расщепление вершины
2
Введите номера вершин ребра для стягивания (например: 1 3): 1 3
Вершины 1 и 3 отождествлены в вершину 1
Ребро (1, 3) стянуто

Результат стягивания ребра:
  1  2  3  4  5  6  7  8  9
1:  1  1  1  1  0  0  1  0  0
2:  1  0  0  0  0  1  1  0  0
3:  1  0  0  0  1  0  1  0  0
4:  1  0  0  0  1  1  1  1  1
5:  0  0  1  1  0  0  0  0  0
6:  0  1  0  1  0  0  1  1  1
7:  1  1  1  1  0  1  0  0  0
8:  0  0  0  1  0  1  0  0  1
9:  0  0  0  1  0  1  0  1  0

```

Задание 2.2

```

Выберите действие: 2

--- Задание 2.1: Унарные операции ---
Создать новый граф или использовать существующий G1?
1. Создать новый случайный граф
2. Использовать граф G1 из задания 1
1
Введите количество вершин: 9

Сгенерированный граф:
  1  2  3  4  5  6  7  8  9
1:  0  0  1  0  0  0  0  0  1
2:  0  0  1  0  1  1  0  0  0
3:  1  1  0  1  1  0  1  0  1
4:  0  0  1  0  0  0  0  1  1
5:  0  1  1  0  0  1  1  0  1
6:  0  1  0  0  1  0  1  0  1
7:  0  0  1  0  1  1  0  0  1
8:  0  0  0  1  0  0  0  0  0
9:  1  0  1  1  1  1  1  0  0

Выберите унарную операцию:
1. Отождествление вершин
2. Стягивание ребра
3. Расщепление вершины
3
Введите номер вершины для расщепления: 4
Вершина 4 расщеплена на вершины 4 и 10

Результат расщепления вершины:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  1  0  0  0  0  0  1  0
2:  0  0  1  0  1  1  0  0  0  0
3:  1  1  0  1  1  0  1  0  1  0
4:  0  0  1  0  0  0  0  0  0  1
5:  0  1  1  0  0  1  1  0  1  0
6:  0  1  0  0  1  0  1  0  1  0
7:  0  0  1  0  1  1  0  0  1  0
8:  0  0  0  0  0  0  0  0  0  1
9:  1  0  1  0  1  1  1  0  0  1
10: 0  0  0  1  0  0  0  1  1  0

```

Задание 2.3

Выберите действие: 3

--- Задание 3.1: Бинарные операции ---
Используются графы G1 и G2 из задания 1
G1:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	0	0	0	0	1	0	0
2:	0	0	1	0	0	0	1	1	0	0
3:	1	1	0	1	1	0	0	0	0	0
4:	0	0	1	0	0	1	0	1	0	0
5:	0	0	1	0	0	1	1	1	1	1
6:	0	0	0	1	1	0	0	0	0	0
7:	0	1	0	0	1	0	0	1	1	1
8:	1	1	0	1	1	0	1	0	0	0
9:	0	0	0	0	1	0	1	0	0	1
10:	0	0	0	0	1	0	1	0	1	0

G2:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	1	0	0	0	1	0	1
2:	0	0	0	1	1	1	1	0	1	0
3:	1	0	0	1	1	0	0	0	1	0
4:	1	1	1	0	0	0	1	1	0	1
5:	0	1	1	0	0	0	1	1	1	1
6:	0	1	0	0	0	0	1	1	0	0
7:	0	1	0	1	1	1	0	0	1	0
8:	1	0	0	1	1	1	0	0	1	0
9:	0	1	1	0	1	0	1	1	0	0
10:	1	0	0	1	1	0	0	0	0	0

Выберите бинарную операцию:

1. Объединение $G = G1 \cup G2$
 2. Пересечение $G = G1 \cap G2$
 3. Кольцевая сумма $G = G1 \oplus G2$
- 1

Результат объединения G1 и G2:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	1	0	0	0	1	0	1
2:	0	0	1	1	1	1	1	1	1	0
3:	1	1	0	1	1	0	0	0	1	0
4:	1	1	1	0	0	1	1	1	0	1
5:	0	1	1	0	0	1	1	1	1	1
6:	0	1	0	1	1	0	1	1	0	0
7:	0	1	0	1	1	1	0	1	1	1
8:	1	1	0	1	1	1	1	0	1	0
9:	0	1	1	0	1	0	1	1	0	1
10:	1	0	0	1	1	0	1	0	1	0

Задание 3.1

Выберите действие: 3

--- Задание 3.1: Бинарные операции ---
Используются графы G1 и G2 из задания 1
G1:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	0	0	0	0	1	0	0
2:	0	0	1	0	0	0	1	1	0	0
3:	1	1	0	1	1	0	0	0	0	0
4:	0	0	1	0	0	1	0	1	0	0
5:	0	0	1	0	0	1	1	1	1	1
6:	0	0	0	1	1	0	0	0	0	0
7:	0	1	0	0	1	0	0	1	1	1
8:	1	1	0	1	1	0	1	0	0	0
9:	0	0	0	0	1	0	1	0	0	1
10:	0	0	0	0	1	0	1	0	1	0

G2:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	1	0	0	0	1	0	1
2:	0	0	0	1	1	1	1	0	1	0
3:	1	0	0	1	1	0	0	0	1	0
4:	1	1	1	0	0	0	1	1	0	1
5:	0	1	1	0	0	0	1	1	1	1
6:	0	1	0	0	0	0	1	1	0	0
7:	0	1	0	1	1	1	0	0	1	0
8:	1	0	0	1	1	1	0	0	1	0
9:	0	1	1	0	1	0	1	1	0	0
10:	1	0	0	1	1	0	0	0	0	0

Выберите бинарную операцию:

1. Объединение $G = G1 \cup G2$
 2. Пересечение $G = G1 \cap G2$
 3. Кольцевая сумма $G = G1 \oplus G2$
- 2

Результат пересечения G1 и G2:

	1	2	3	4	5	6	7	8	9	10
1:	0	0	1	0	0	0	0	1	0	0
2:	0	0	0	0	0	0	1	0	0	0
3:	1	0	0	1	1	0	0	0	0	0
4:	0	0	1	0	0	0	0	1	0	0
5:	0	0	1	0	0	0	1	1	1	1
6:	0	0	0	0	0	0	0	0	0	0
7:	0	1	0	0	1	0	0	0	1	0
8:	1	0	0	1	1	0	0	0	0	0
9:	0	0	0	0	1	0	1	0	0	0
10:	0	0	0	0	1	0	0	0	0	0

Задание 3.2

```

Выберите действие: 3

--- Задание 3.1: Бинарные операции ---
Используются графы G1 и G2 из задания 1
G1:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  1  0  0  0  0  1  0  0
2:  0  0  1  0  0  0  1  1  0  0
3:  1  1  0  1  1  0  0  0  0  0
4:  0  0  1  0  0  1  0  1  0  0
5:  0  0  1  0  0  1  1  1  1  1
6:  0  0  0  1  1  0  0  0  0  0
7:  0  1  0  0  1  0  0  1  1  1
8:  1  1  0  1  1  0  1  0  0  0
9:  0  0  0  0  1  0  1  0  0  1
10: 0  0  0  0  1  0  1  0  1  0

G2:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  1  1  0  0  0  1  0  1
2:  0  0  0  1  1  1  1  0  1  0
3:  1  0  0  1  1  0  0  0  1  0
4:  1  1  1  0  0  0  1  1  0  1
5:  0  1  1  0  0  0  1  1  1  1
6:  0  1  0  0  0  0  1  1  0  0
7:  0  1  0  1  1  1  0  0  1  0
8:  1  0  0  1  1  1  0  0  1  0
9:  0  1  1  0  1  0  1  1  0  0
10: 1  0  0  1  1  0  0  0  0  0

Выберите бинарную операцию:
1. Объединение  $G = G1 \cup G2$ 
2. Пересечение  $G = G1 \cap G2$ 
3. Кольцевая сумма  $G = G1 \oplus G2$ 
3

Результат кольцевой суммы G1 и G2:
  1  2  3  4  5  6  7  8  9 10
1:  0  0  0  1  0  0  0  0  0  1
2:  0  0  1  1  1  1  0  1  1  0
3:  0  1  0  0  0  0  0  0  1  0
4:  1  1  0  0  0  1  1  0  0  1
5:  0  1  0  0  0  1  0  0  0  0
6:  0  1  0  1  1  0  1  1  0  0
7:  0  0  0  1  0  1  0  1  0  1
8:  0  1  0  0  0  1  1  0  1  0
9:  0  1  1  0  0  0  0  1  0  1
10: 1  0  0  1  0  0  1  0  1  0

```

Задание 3.3

Вывод:

В ходе выполнения данной лабораторной работы была успешно реализована программа для освоения методов программной обработки унарных и бинарных операции над графами.